Course End Project

Panic Button Emergency System using Arduino

IOT LABORATORY (A8603)

Bachelor of Technology

in

COMPUTER SCIENCE AND ENGINEERING

Submitted By

24881A05A4 K .Jahnavi
24881A05C7 V. Vijayasree

Under the Guidance of

MS BAALNE ANJALI
Assistant Professor, Dept of CSE

Department of Computer Science and Engineering

VARDHAMAN COLLEGE OF ENGINEERING

(AUTONOMOUS)

Affiliated to JNTUH, Approved by AICTE, Accredited by NAAC with A++ Grade,
UGC "College with Potential for Excellence" and ISO 9001:2015 Certified

Shamshabad - 501 218, Hyderabad

2025 - 2026

VARDHAMAN
COLLEGE OF ENGINEERING

Autonomous Institute, Affiliated to **JNTUH,** Approved by **AICTE,** Accredited by **NAAC** with **A++** Grade
Kacharam, Shamshabad, Hyderabad - 501 218, Telangana, India.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

This is to certify that the Course End Project titled **"Panic Button Emergency System using Arduino"** is carried out by **K.Jahnavi, V.Vijayasree** with Roll Numbers **24881A05A4, 24881A05C7** towards **A8603 - IOT Laboratory** course in partial fulfilment of the requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** during the Academic year 2025-26.

Signature of the Course Faculty	**Signature of the HOD**
Ms Baalne Anjali	**Dr. Karnati Ramesh**
Assistant Professor, CSE	Assistant Professor and Head, CSE

VARDHAMAN COLLEGE OF ENGINEERING

(AUTONOMOUS)
Affiliated to JNTUH, Approved by AICTE, Accredited by NAAC with A++ Grade,
UGC "College with Potential for Excellence" and ISO 9001:2015 Certified

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Course End Project
IOT Laboratory (A8603)
Check List

S. No.	Content	Status (✓ / ×)
1	Abstract	✓
2	Objectives	✓
3	Problem Statement	✓
4	Algorithm	✓
5	Flow Chart	✓
6	Source Code	✓
7	Input	✓
8	Output	✓
9	Conclusion	✓
10	Future Scope	✓
11	References	✓

Signature of the Guide
Ms. Baalne Anjali
Assistant Professor
Dept. of CSE

Course End Project
IOT Laboratory (A8603)
Table of Content

S. No.	Content	Page. No.
1	Abstract	5
2	Problem Statement	6
3	Objectives	7
4	Algorithm	8
5	Flow Chart	9-12
6	Source Code	13
7	Input	14
8	Output	15
9	Conclusion	16
10	Future Scope	17
11	References	18

1. ABSTRACT

The Panic Button Emergency System is a simple yet effective safety device built using an Arduino Uno microcontroller. The system is designed to provide an immediate alert response when a person is in danger or requires urgent assistance. When the push button is pressed, the buzzer is activated instantly, producing a loud alert sound to draw attention from nearby people.

This project demonstrates how basic electronic components such as a push button, buzzer, jumper wires, and a breadboard can be combined with an Arduino Uno to build a practical emergency alert mechanism. The system is low-cost, easy to set up, and highly responsive, making it suitable for use in homes, schools, hospitals, and other environments where quick emergency alerts are needed.

The project is implemented as part of the IoT Laboratory course and provides hands-on experience with digital input/output operations on the Arduino platform. The code is written in the Arduino programming language (C/C++ based) and uses simple logic to continuously monitor the button state and trigger the buzzer accordingly.

Keywords:
Panic Button, Emergency Alert System, Arduino Uno, Push Button, Buzzer, IoT Project, Safety Device, Embedded Systems, Digital Input/Output, Real-Time Alert, Low-Cost System, Rapid Response, Breadboard Circuit, Arduino Programming, Smart Safety Solution

2. PROBLEM STATEMENT

In many real-world situations, people who are in danger or need immediate help may not have the time or ability to communicate through traditional means such as phone calls or shouting for help. Elderly people living alone, patients in hospitals, or individuals in unsafe environments often face situations where they need to call for help instantly.

Existing emergency systems can be expensive, complex, or require internet connectivity to function, making them inaccessible for common use. There is a need for a simple, affordable, and reliable emergency alert system that anyone can use without technical knowledge.

This project addresses this problem by designing a basic Panic Button Emergency System using an Arduino Uno that can trigger an immediate audio alert (via a buzzer) when a button is pressed. The system is cost-effective, easy to use, and does not require any internet connection or smartphone, making it highly practical for everyday emergency scenarios.

3. OBJECTIVES

The main objectives of this project are:

1. To design and implement a simple emergency alert system using Arduino Uno.
2. To interface a push button with the Arduino for detecting an emergency trigger.
3. To activate a buzzer as an audio alert when the panic button is pressed.
4. To display status messages on the Serial Monitor for debugging and monitoring purposes.
5. To understand the working of digital input/output in Arduino microcontroller programming.
6. To develop a low-cost, portable, and reliable emergency alert system suitable for real-world use.
7. To gain practical experience with breadboard circuit assembly and Arduino hardware interfacing.

4. ALGORITHM

The following step-by-step algorithm describes the working of the Panic Button Emergency System:

- START: Begin the program execution.
- INITIALIZE: Set up the push button pin (Pin 2) as INPUT and the buzzer pin (Pin 8)as OUTPUT. Initialize Serial communication at 9600 baud rate.
- MONITOR: Continuously read the digital state of the push button using the digitalRead() function.
- CHECK CONDITION: If the button state reads HIGH (button pressed), proceed to Step 5. If the button state reads LOW (button not pressed), proceed to Step 7.
- ACTIVATE BUZZER: Send a HIGH signal to the buzzer pin to activate it and produce an alert sound.
- SEND ALERT: Print "EMERGENCY ALERT! Panic Button Pressed!" on the Serial Monitor.
- WAIT: Keep the buzzer active for 3 seconds (3000 milliseconds delay) for sufficient alerting.
- DEACTIVATE BUZZER: Send a LOW signal to the buzzer pin to turn it off when button is not pressed.
- REPEAT: Apply a short delay of 100 ms for debounce, then go back to Step 3 and continue monitoring.
- END: The system keeps running in an infinite loop until power is disconnected.

5. PROPOSED SYSTEM

The proposed system is a hardware-software integrated emergency alert solution built around the Arduino Uno microcontroller. The system consists of the following components working together: a push button serves as the panic trigger, a buzzer acts as the audio alert output device, and jumper wires and a breadboard are used to make the electrical connections.

The system works in real-time. As soon as the push button is pressed, the Arduino detects the HIGH signal on its digital input pin and immediately activates the buzzer by sending a HIGH signal to the output pin. The buzzer produces a loud and continuous beeping sound for 3 seconds to alert people in the surrounding area. After 3 seconds, the system resets and goes back to the monitoring state.

The hardware is assembled on a breadboard, making it easy to modify and test. The system is powered through the Arduino USB connection or a battery pack, making it portable and easy to deploy in various locations.

5.1 Flow Chart

The following flowchart illustrates the complete working process of the Panic Button Emergency System:

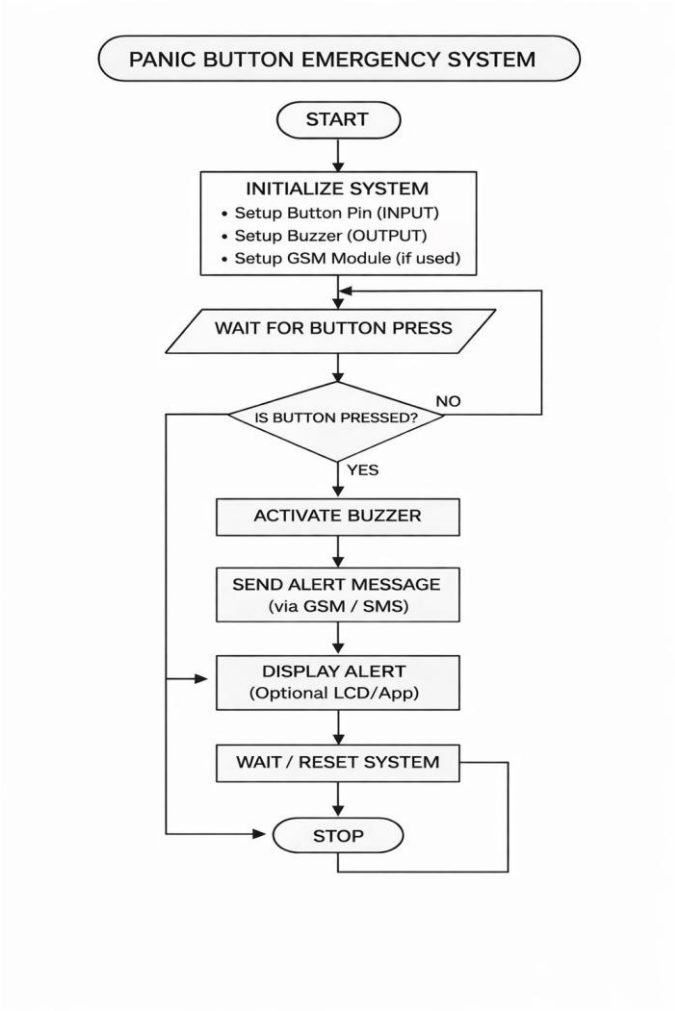


Figure 1: Flowchart of Panic Button Emergency System

5.2 Working Model

Objectives

The main objective of this project is to design and develop a simple Panic Button Emergency System using Arduino Uno. The system aims to provide an immediate alert during emergency situations. It focuses on creating a low-cost, reliable, and easy-to-use safety device that does not require internet connectivity. Additionally, the project helps in understanding the basic concepts of embedded systems, digital input/output operations, and Arduino programming.

Components Required

The system uses basic electronic components that are easily available and cost-effective.

Hardware Components:

- Arduino Uno
- Push Button
- Buzzer
- Breadboard
- Jumper Wires
- Resistor (10kΩ)
- USB Cable

Software Requirements:

- Arduino IDE
- Arduino Programming (C/C++ based)

System Workflow

The system starts working when power is supplied to the Arduino Uno. The microcontroller initializes the input and output pins, where the push button acts as an input device and the

buzzer acts as an output device. The Arduino continuously monitors the state of the push button.

When the button is not pressed, the system remains idle and the buzzer stays OFF. When the user presses the panic button, the Arduino detects the change in input signal and immediately activates the buzzer. The buzzer produces a loud sound to alert nearby people. Once the button is released, the buzzer turns OFF, and the system returns to its normal state. This process continues in a loop, ensuring real-time response.

Implementation Steps

1. Connect the push button to a digital input pin of Arduino.
2. Connect the buzzer to a digital output pin.
3. Use a resistor to ensure proper input signal (pull-up/pull-down).
4. Assemble the circuit on a breadboard using jumper wires.
5. Write the Arduino code to read button input and control the buzzer.
6. Upload the code to Arduino using Arduino IDE.
7. Power the system and test by pressing the button.

Advantages

- Simple and easy to design
- Low cost and affordable
- Provides instant alert response
- Does not require internet connectivity
- Easy to use for all age groups
- Reliable and fast operation

:

6. SOURCE CODE

```
const int buttonPin = 2;
const int buzzerPin = 8;

int buttonState = 0;
int lastButtonState = 0;

void setup() {
  pinMode(buttonPin, INPUT);
  pinMode(buzzerPin, OUTPUT);
  Serial.begin(9600);
}

void loop() {
  buttonState = digitalRead(buttonPin);

  // Detect button press only once (edge detection)
  if (buttonState == HIGH && lastButtonState == LOW) {
    Serial.println(" EMERGENCY ALERT! Panic Button Pressed!");

    digitalWrite(buzzerPin, HIGH);
    delay(3000);  // buzzer ON for 3 sec
    digitalWrite(buzzerPin, LOW);

    Serial.println("System Reset. Monitoring...");
  }

  lastButtonState = buttonState;
  delay(50); // debounce delay
}
```

7. INPUT

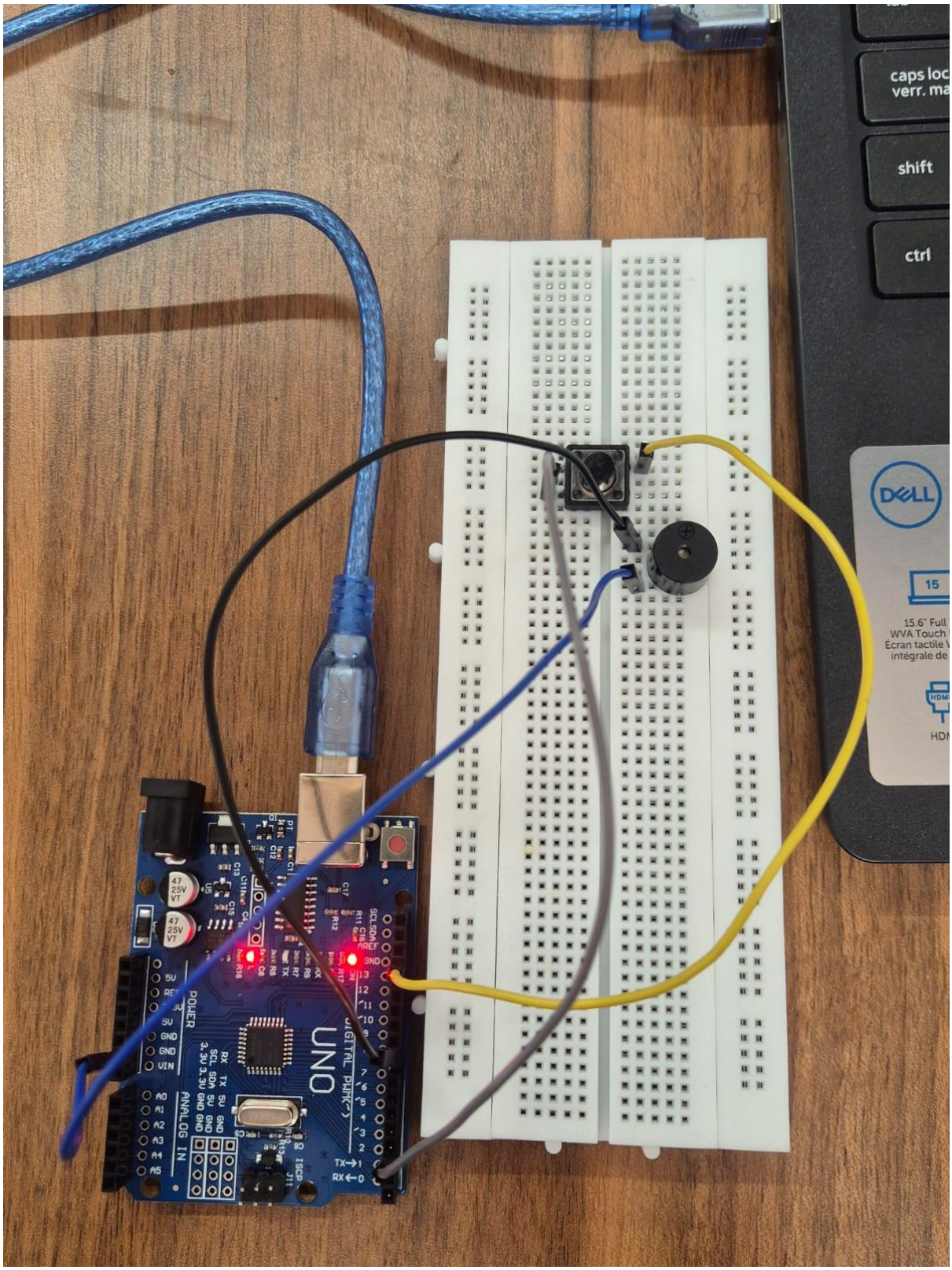
The input to the Panic Button Emergency System is provided through the physical push button connected to digital pin D2 of the Arduino Uno. The following describes the input mechanism:

1. Physical Input: The user presses the push button physically. This action creates an electrical connection and sends a HIGH (5V) signal to the Arduino input pin.
2. Digital Signal: The Arduino reads the digital input from pin D2. When the button is pressed, the pin reads a logic HIGH (1). When not pressed, it reads logic LOW (0).
3. Button Debounce: A 100ms software delay is included to prevent false triggers due to mechanical bouncing of the button contacts.
4. Power Input: The Arduino is powered via a 5V USB connection from a computer or an external 9V battery adapter.

Input Summary Table:

Input Type	Source	Arduino Pin
Digital Input	Push Button (when pressed)	D2 (HIGH signal = 5V)
Digital Input	Push Button (not pressed)	D2 (LOW signal = 0V)
Power Input	USB or Battery	VIN / USB Port (5V/9V)

8. OUTPUT


caps loc
verr. ma
shift
ctrl
DELL
15
15.6" Full
WVA Touch
Écran tactile
intégrale de
HDM
UNO
POWER
ANALOG IN
DIGITAL PWM(~)

9. CONCLUSION

The Panic Button Emergency System using Arduino has been successfully designed, built, and tested. The system demonstrates a practical and affordable approach to emergency alert systems using readily available electronic components. By pressing a single button, the system immediately activates a buzzer to generate an audio alert, which can draw the attention of nearby people in emergency situations.

This project provided valuable hands-on experience with Arduino microcontroller programming, digital input/output operations, hardware interfacing, and breadboard circuit assembly. The simplicity of the design ensures that the system is easy to replicate, modify, and expand based on future requirements.

The project successfully achieved all its objectives: the push button was interfaced as a digital input, the buzzer was controlled as a digital output, and the Serial Monitor displayed appropriate status messages. The system was found to be stable, reliable, and responsive during testing.

In conclusion, the Panic Button Emergency System is a useful and practical IoT project that can be deployed in real-world scenarios to improve personal safety. It serves as a solid foundation for building more advanced emergency alert and monitoring systems in the future.

10. FUTURE SCOPE

The current implementation of the Panic Button Emergency System can be significantly enhanced in the future to make it more powerful and useful. Some potential improvements and extensions include:

1. GSM/SMS Alert Integration: By adding a GSM module (such as the SIM800L), the system can send an automatic SMS alert to a pre-configured emergency contact number when the panic button is pressed. This extends the reach of the alert beyond the immediate surroundings.
2. Wi-Fi / IoT Connectivity: Using an ESP8266 or ESP32 module, the system can be connected to the internet to send real-time notifications via email, WhatsApp, or IoT platforms like ThingSpeak or Blynk.
3. GPS Location Tracking: Adding a GPS module can enable the system to send the exact geographic location of the user along with the emergency alert, which is useful for outdoor safety and elderly tracking applications.
4. LCD/LED Status Display: An LCD display or LED indicators can be added to show the current system status, battery level, and alert history to the user.
5. Wearable Design: The system can be miniaturized and packaged into a wearable device such as a wristband or pendant, making it more convenient for elderly people, children, or patients.
6. Multiple Sensor Integration: Additional sensors such as fall detection sensors, heart rate monitors, or gas sensors can be integrated to create a comprehensive health and safety monitoring system.
7. Cloud Dashboard: The system data can be logged and visualized on a cloud dashboard, allowing caregivers or security personnel to monitor alert history and system status remotely.

11. REFERENCES

[1] Arduino Official Documentation - Arduino Uno Reference Manual. Available at: https://www.arduino.cc/en/Reference/HomePage

[2] Blum, J. (2013). Exploring Arduino: Tools and Techniques for Engineering Wizardry. Wiley Publications.

[3] Monk, S. (2016). Programming Arduino: Getting Started with Sketches. McGraw-Hill Education.

[4] McRoberts, M. (2010). Beginning Arduino. Apress Publications.

[5] Banzi, M. & Shiloh, M. (2014). Getting Started with Arduino: The Open Source Electronics Prototyping Platform. Maker Media.

[6] Instructables.com - Arduino Push Button and Buzzer Tutorial. Available at: https://www.instructables.com/Arduino-Button-and-Buzzer/